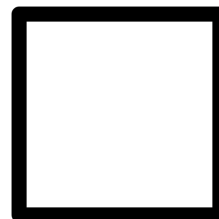


Programação e Desenvolvimento de Software 1 — Prova 3

Nome: _____

Matrícula UFMG: _____

Assinatura: _____



Instruções:

- Todas as questões devem ser resolvidas utilizando a linguagem C.
- Se necessário, utilize a folha de papel almaço como rascunho para construir a solução. Transcreva a solução final para a prova no espaço reservado para cada questão.
- Termo de consentimento e ética: ao assinar esta prova declaro que usei somente o material permitido e que estou ciente de que *qualquer identificação de cola ou uso de celular durante a avaliação* determinará uma nota zero e encaminhamento de meu nome para processo disciplinar discente junto à Diretoria de sua Unidade. O celular deve estar desligado!

Função/Operador	Descrição	Exemplo
<code>void* malloc (size_t size)</code>	Aloca um bloco de memória de tamanho size, retornando um ponteiro para o início do bloco.	<code>int *p1 = (int*)malloc(sizeof(int));</code>
<code>void free(void *ptr)</code>	Libera um bloco de memória apontado por ptr.	<code>free(ptr);</code>
<code>FILE* fopen(const char *filename, const char *mode)</code>	Abre o arquivo filename no modo mode.	<code>FILE *temp = fopen("temp.txt", "w");</code>
<code>int fclose(FILE *arq)</code>	Fecha o arquivo arq.	<code>fclose(arq);</code>
<code>int fscanf(FILE *arq, const char *format, &variaveis)</code>	Lê dados formatados do arquivo arq.	<code>fscanf(arq, "%s %f", nome, &nota1);</code>

Parte 1 – Simple Fighter (23 pontos)

Nas questões 1–4 iremos fazer uma versão simplificada do jogo Auto Chess, onde lutadores brigam de forma automática em uma arena. Nas quatro questões iremos (1) carregar informações dos lutadores, (2) calcular o dano infligido por um lutador ao atacar o inimigo e (3) combinar todas as funções para simular uma luta. Os lutadores serão representados pela seguinte estrutura:

```
struct lutador {
    char *nome;
    int vitalidade;
    int dano_maximo;
};
typedef struct lutador Lutador;
```

Além disso, você deve utilizar a função abaixo para gerar um número inteiro aleatório em um intervalo:

```
int randint(int min, int max) {  
    return min + rand()%(max-min+1);  
}
```

A função `main()` deste programa é:

```
int main() {  
    srand(time(NULL));  
    char resp;  
    do {  
        batalha();  
        printf("\nDe novo? (S/N)");  
        fflush(stdin);  
        resp = getchar();  
    } while(toupper(resp) == 'S');  
    return 0;  
}
```

Considere o arquivo de entrada `lutadores.txt` com o formato abaixo, onde a primeira linha contém o número de lutadores e as demais o nome, vitalidade e dano máximo de cada lutador:

```
3  
Ryu 100 8  
Ken 90 10  
ChunLi 80 12
```

Questão 1 – Carregamento de Lutador (7 pontos)

Implemente a função `carrega_lutadores(int *n)` que lê os dados dos lutadores a partir do arquivo `lutadores.txt`, salva o número de lutadores a serem lidos, aloca dinamicamente um vetor de `Lutador` e o preenche com os dados lidos.

A função deve seguir os seguintes passos:

1. Abrir o arquivo `lutadores.txt` para leitura.
2. Ler o número de lutadores, que está na primeira linha do arquivo, e armazenar no endereço apontado por `n`.
3. Alocar dinamicamente memória para armazenar `n` lutadores.
4. Preencher o vetor com os dados de cada lutador lidos do arquivo.
5. Retornar o ponteiro para o vetor de lutadores.

Você pode supor que os nomes dos lutadores vão ter **no máximo 10 caracteres**.

```
Lutador* carrega_lutadores(int *n) {
```

```
}
```

Questão 2 – Cálculo do Dano (3 pontos)

Quando um lutador ataca outro, ele causa uma quantidade de dano **aleatório** uniformemente distribuído entre 1 e seu **dano_maximo**. Ao receber um ataque, o dano do ataque é subtraído da vitalidade do lutador.

Implemente um procedimento (função **void**) chamado **ataque** que recebe como parâmetros dois lutadores. Para fins de clareza, os parâmetros devem ter nome **atacante** e **alvo**. Sua função deve calcular o dano causado pelo **atacante** e atualizar a vitalidade do **alvo**.

Questão 3 – Elimina lutador (4 pontos)

Implemente uma função que verifica se um lutador **ferido** está rendido (isto é, com vitalidade menor ou igual a zero). Em caso positivo, primeiro libere qualquer memória alocada dinamicamente para o lutador. Em seguida, atualize o vetor de lutadores **v[]** de forma que os lutadores que ainda estão ativos (isto é, com vitalidade maior que zero) fiquem armazenados no início do vetor **v[]** (isto é, nas posições de zero a **n-1**). Por fim, atualize a quantidade de lutadores ativos na variável **n** recebida por referência. **Importante:** Não é necessário realocar a memória de **v[]** usando **realloc()**, etc.

A função deve receber:

- o vetor de lutadores **v[]**;
- o índice **ferido** do lutador que sofreu dano;
- e um ponteiro para a variável **n**, que representa a quantidade atual de lutadores vivos no vetor.

```
void eliminaLutador(Lutador v[], int ferido, int *n) {  
  
    if(_____) {  
  
        _____;  
  
        _____;  
  
        _____;  
  
    }  
}
```

Questão 4 – Simulador de Luta (9 pontos)

Complete as lacunas abaixo utilizando as funções desenvolvidas nas questões anteriores para implementar a função **batalha** do programa. Sua função deve:

1. Carregar os lutadores usando a função **carregaLutadores**.
2. Simular rodadas de ataques entre os lutadores. Em cada iteração do laço, dois lutadores devem ser escolhidos de forma aleatória, sendo que o primeiro sorteado ataca o segundo.
3. Depois do ataque, verifique se o lutador que sofreu dano morreu e atualize o vetor de lutadores. Use a função **eliminaLutador**.
4. A batalha termina quando restar apenas um lutador. Imprima na tela o nome desse lutador.
5. Liberar qualquer memória alocada dinamicamente.

```

void batalha() {

    int n, l1, l2;

    Lutador *lutadores = _____; //1

    while(_____) { //1

        l1 = randint(_____); //0.5

        l2 = randint(_____); //0.5
        if(l1 != l2) {

            _____; //1

            _____; //1

            if(_____) //1

                printf("\nVencedor:%s", _____); //1
        }
    }

    _____; //1

    _____; //1
}

```

Questão 5 – Aproximação de raiz quadrada (7 pontos)

Implemente uma função **recursiva** chamada **raizb** que calcule uma aproximação da raiz quadrada utilizando o **método babilônico** a partir de **n** passos. Sua função deve ter o seguinte protótipo:

```
double raizb(double x, int n);
```

onde **x** é um valor inteiro positivo cuja raiz quadrada queremos calcular e **n** é o número de passos (iterações) que a função deve executar para calcular a aproximação. A implementação deve ser **totalmente recursiva** (sem **for**, **while** ou variáveis globais). Ao final das **n** chamadas, a função deve retornar a aproximação final. A fórmula do método babilônico é dada por:

- A aproximação inicial é dada por: $a_1 = \frac{x}{2}$.
- A aproximação depois de **n** passos (iterações) é dada por: $a_n = \frac{a_{n-1} + \frac{x}{a_{n-1}}}{2}$.

Exemplo: se $x = 9$ e $n = 2$, a sua função deve retornar 3.25, pois $a_2 = \frac{a_1 + 9/a_1}{2}$, $a_1 = 9/2$ e $a_2 = 3.25$.

GABARITO:

Questão 1

```
Lutador* carrega_lutadores(int *n) {
    FILE *arquivo = fopen("lutadores.txt", "r");
    if (arquivo == NULL) {
        printf("Erro ao abrir o arquivo.\n");
        return NULL;
    }
    fscanf(arquivo, "%d", n);
    Lutador *vetor = (Lutador*) malloc(*n * sizeof(Lutador));
    if (vetor == NULL) {
        printf("Erro de alocação de memória.\n");
        fclose(arquivo);
        return NULL;
    }
    for (int i = 0; i < *n; i++) {
        vetor[i].nome = (char *) malloc(10 * sizeof(char));
        fscanf(arquivo, "%s %d %d",
                vetor[i].nome,
                &vetor[i].vitalidade,
                &vetor[i].dano_maximo);
    }
    fclose(arquivo);
    return vetor;
}
```

Questão 2

```
void ataque(Lutador atacante, Lutador *alvo) {
    // Calcula o dano aleatório entre 1 e dano_maximo do atacante
    int dano = inteiro_aleatorio(1, atacante.dano_maximo);

    // Subtrai o dano da vitalidade do alvo
    alvo->vitalidade -= dano;

    // Garante que a vitalidade não fique negativa (opcional)
    if (alvo->vitalidade < 0) {
        alvo->vitalidade = 0;
    }
}
```


Questão 3

```
void eliminaLutador(Lutador v[], int ferido, int *n) {
    if(v[ferido].vitalidade <=0) {
        free(v[ferido].nome);
        v[ferido] = v[*n-1];
        *n = *n - 1;
    }
}
```

Questão 4

```
void batalha() {

    int n, l1, l2;
    Lutador *lutadores = carrega_lutadores(&n);
    while(n>1) {
        l1 = randint(0,n-1);
        l2 = randint(0,n-1);
        if(l1 != l2) {
            ataque(lutadores[l1], &lutadores[l2]);
            eliminaLutador(lutadores, l2, &n);
            if(n==1)
                printf("\nVencedor: %s", lutadores[0].nome);
        }
    }
    free(lutadores[0].nome);
    free(lutadores);
}
```

Questão 5

```
double raizb(int x, int n) {
    if(n == 1) return x/2.0;
    double prev = raizb(x, n-1);
    return 0.5 * (prev + x/prev);
}

double raizb(int x, int n) {
    if(n == 1) return x/2.0;
    return (raizb(x, n-1) + x/raizb(x, n-1)) / 2;
}
```